How to Read GPU Kernel Traces at the Hardware Level

Sub-title : A low-level investigation of GPU kernel execution on AMD MI300X

When an application is slow, the application can usually tell us **that** something is taking time. It does not always tell us **why**.

For an LLM serving workload, vLLM can report request latency, throughput, token generation speed, GPU utilization, and other application-level metrics. Those numbers are useful, but they do not tell us which GPU kernels ran, how often they ran, how much time they consumed, or what the GPU was doing while those kernels executed.

That is where `rocprofv3` becomes useful.

`rocprofv3` is AMD's command-line GPU profiling tool. It shows what happens below the application layer, directly on the GPU hardware. For anyone working with GPUs or low-level performance, it is the gold-standard tool to get familiar with. It plays a similar role to NVIDIA's Nsight profiling tools.

The goal is not simply to collect profiler output. It is to use that output to build a mental model of GPU execution: which kernels run, how they are launched, how much work they perform, and how much data they move.

What is rocprofv3?

It helps answer questions such as:

- Which GPU kernels are running?
- How are kernels launched?
- Which kernels consume the most GPU time?
- How many times are they launched?
- How much arithmetic do they perform?
- How much data do they move?

The important point is that rocprofv3 does not optimize the application for you. It gives you the evidence to make those decisions.

Profiling process image : place a visual

I used this process on Qwen3-30B-A3B inference workload running through vLLM on an AMD MI300X. A single request produced tens of thousands of GPU dispatches. The goal was to reduce that activity to one question: which kernel deserves attention, and what is the GPU actually doing while it runs?

1. Define the workload

Before profiling, I needed a repeatable workload.

The experiment used:

Item	Configuration
GPU	AMD MI300X
Software	ROCm 10.0, vLLM 0.27.0
Model	Qwen3-30B-A3B
Precision	BF16
Input	One fixed prompt
Batch size	1

Output	Exactly 64 generated tokens
Inference path	`vLLM LLM.generate()`
Warm-up	8 generated tokens before measurement

The Python program used vLLM’s offline inference interface; there was no HTTP API server in this path.

Python script

↓

LLM.generate()

↓

vLLM inference engine

↓

ROCm

↓

MI300X

The model was loaded before the measured request began. The script ran a short warm-up, synchronized the GPU, and only then started profiling.

The measurement boundary was:

Load model

↓

Warm up

↓

GPU synchronize

↓

PROFILER RESUME

↓

Generate exactly 64 tokens

↓

GPU synchronize

↓

PROFILER PAUSE

The script used profiler resume and pause controls to keep model loading, warm-up, compilation, initialization, and graph setup outside the measured inference region.

Now there was a controlled workload.

What actually ran on the GPU?

2. Capture the first trace

The first profiling pass was a kernel trace. I did not start by collecting hardware counters—GPU-internal measurements such as instruction counts, memory stalls, cache activity, and compute-unit utilization. Those are useful for explaining *why* a kernel is slow, but first I needed to know *which* kernels deserved that closer look.

At this stage, I only wanted to know:

- What ran?
- How often?
- How long?

A simplified version of the tracing command was:

```
rocprofv3 \
  --kernel-trace \
  --stats \
  --output-format csv \
  --selected-regions \
  -- python scripts/mix300-device/vllm-kernel-64tok.py
```

The important output files were:

1_kernel_trace.csv - every individual GPU dispatch

1_kernel_stats.csv - aggregated time and execution count by kernel

The trace contained 54,194 GPU kernel dispatches. That number alone was not useful. The next step was to find which launches deserved a closer look.

3. Find `fused_moe_kernel`

The aggregated kernel statistics gave the first useful view of the workload. The largest entries included:

Kernel	Executions	Total duration	Share of summed kernel duration
`fused_moe_kernel`	6,144	85.539 ms	25.72%
`wvSplitK…`	6,112	51.558 ms	15.50%
`topkGating`	3,072	25.118 ms	7.55%
`paged_attention…`	3,072	20.896 ms	6.28%
`reshape_and_cache…`	3,072	14.279 ms	4.29%

The raw kernel summary reported `fused_moe_kernel` at 6,144 calls, 85,539,433 ns of accumulated duration, and 25.72% of summed kernel time.

```
54,194 GPU dispatches

          ↓

6,144 fused_moe_kernel dispatches

          ↓

25.72% of summed kernel duration
```

This does not mean `fused_moe_kernel` was responsible for exactly 25.72% of end-to-end request latency. CPU activity, gaps between GPU work, synchronization, and overlapping work can all affect request latency.

But it does identify an important target: this kernel family is worth investigating.

That is the purpose of the first trace. It shows where to look. It does not yet explain why the kernel costs what it costs.

4. Inspect the launch pattern

Step 3 identified `fused_moe_kernel` as an important target, but it did not explain how its 6,144 launches were organized. This step examines the launch pattern to see whether the launches map to the model's structure: its layers, initial pass, and repeated decode steps. The question changes from "which kernel is expensive?" to "which part of the model's execution is producing that cost?"

The next step was to stop treating all 6,144 launches as identical. Within the fused_moe_kernel family, the trace contained four kernel IDs:

Kernel ID	Dispatches	Grid size	Workgroup size	Average trace duration
9119	48	98,304	256	25.395 µs
9120	48	131,072	256	13.688 µs
9121	3,024	49,152	256	17.839 µs
9122	3,024	65,536	256	9.827 µs

All four used the same resources:

```
VGPRs   = 80

SGPRs   = 48

LDS     = 0

Scratch = 0
```

MI300X uses 64-thread wavefronts. A 256-thread workgroup therefore contains:

256/64 = 4 wavefronts

```
1 workgroup

    ↓

256 threads

    ↓

4 wavefronts
```

The execution counts revealed more about the workload. Qwen3-30B-A3B has 48 layers, and the measured request generated exactly 64 tokens. The first model pass predicts the first token; the remaining 63 tokens require 63 decode steps.

For kernel IDs 9121 and 9122:

3,024=63×48

The launch pattern therefore fits:

```
Initial model step

48 × kernel 9119

48 × kernel 9120
```

↓

Repeated decode

63 decode steps

× 48 layers

× kernel 9121

× kernel 9122

The totals also match:

48+48+3,024+3,024=6,144

The trace timestamps placed all 9119 and 9120 launches before the repeated 9121 and 9122 sequence. The profiler was no longer showing only a list of kernel names; it was revealing how model execution repeated across layers and decode steps.

The repeated decode kernels—9121 and 9122—became the next target.

5. Isolate the kernel

Detailed profiling has overhead. There was no reason to collect hardware counters for every kernel in the request. The kernel family of interest was already known, so the counter configuration filtered for:

kernel_include_regex: "^fused_moe_kernel$"

The application is still executed normally. The filter changed what the profiler measured, not what vLLM executed.

This is an important profiling habit: start broad with tracing, then narrow the measurement once the important kernels are known.

The investigation had narrowed from:

Whole application

↓

54,194 dispatches

↓

fused_moe_kernel

↓

6,144 dispatches

↓

9121 + 9122 repeated decode kernels

Now the question could become much more precise: **what is causing the repeated decode kernels to cost what they cost?**

6. Choose counters based on a question

Hardware profilers expose many counters. Collecting all of them is not a profiling strategy. The counters should follow from a specific question.

For the repeated decode kernels, the question was:

How much BF16 matrix arithmetic does the kernel execute, and how much data does it move?

The counters answer two related questions:

- **How much useful work does the kernel do?** `SQ_INSTS_VALU_MFMA_MOPS_BF16` measures BF16 matrix arithmetic.
- **How much data does it move?** `FETCH_SIZE` and `WRITE_SIZE` measure reads from and writes to GPU HBM memory.

Together, they help distinguish a compute-bound kernel from a memory-bound kernel.

Counter	What it measures
`SQ_INSTS_VALU_MFMA_MOPS_BF16`	**BF16 matrix arithmetic operations**
`FETCH_SIZE`	**Data read from GPU HBM memory**
`WRITE_SIZE`	**Data written to GPU HBM memory**

The configuration used separate performance-monitoring groups because the GPU cannot collect every requested counter at the same time. Requesting them together failed with:

`ERROR : Request exceeds capabilities of hardware to collect`

`rocprofv3` therefore rotates the counter groups across repeated kernel dispatches. Because kernels 9121 and 9122 repeat with the same shape and workload, each counter group covers a representative subset of launches:

```
jobs:
  - pmc_groups:
      - ["SQ_INSTS_VALU_MFMA_MOPS_BF16"]
      - ["FETCH_SIZE"]
      - ["WRITE_SIZE"]
```

```
pmc_group_interval: 8

kernel_include_regex: "^fused_moe_kernel$"
```

Kernels 9121 and 9122 each ran 3,024 times. The three counter groups each collected 1,008 samples per kernel, with one counter measurement recorded per dispatch.

1,008×3=3,024

That matches the total number of launches for each kernel and confirms balanced sampling across the repeated decode series.

This matters because the counters were collected in separate groups across different kernel launches. Before comparing their averages, confirm that each counter group sampled the same repeated decode workload.

7. Convert counters into FLOPs and bytes

Raw hardware-counter values are not the end result. They need to be converted into quantities that explain the workload.

For the steady decode kernels, the counter averages were:

Measurement	Kernel 9121	Kernel 9122

FETCH_SIZE	49,195.323 KB	24,707.041 KB
WRITE_SIZE	24 KB	32 KB
MFMA counter	1,572,864	786,432
MFMA-sample duration	18.026 µs	9.894 µs

Matrix arithmetic

For this BF16 MFMA counter, each unit corresponds to 512 floating-point operations.

For kernel 9121:

$$1{,}572{,}864 \times 512 = 805{,}306{,}368$$

One 9121 dispatch executed approximately:

805.31 million BF16 FLOPs

For kernel 9122:

$$786{,}432 \times 512 = 402{,}653{,}184$$

One 9122 dispatch executed approximately:

402.65 million BF16 FLOPs

Data movement

The `FETCH_SIZE` counter measures HBM reads and `WRITE_SIZE` measures HBM writes, both in KB. Adding them gives the total HBM traffic for one dispatch:

Kernel	HBM reads	HBM writes	Total HBM traffic
9121	49,195.323 KB	24 KB	**≈ 49.22 MB**
9122	24,707.041 KB	32 KB	**≈ 24.74 MB**

Arithmetic intensity

Arithmetic intensity compares the work performed with the data moved:

$$Arithmetic\ intensity = \frac{FLOPs}{Bytes}$$

Kernel	Executed BF16 FLOPs	Total HBM traffic	Arithmetic intensity
9121	805.31 million	49.22 MB	**≈ 16.36 FLOP/B**
9122	402.65 million	24.74 MB	**≈ 16.28 FLOP/B**

Combined result

Quantity	Kernel 9121	Kernel 9122	Combined
Executed BF16 work	805.31 million FLOPs	402.65 million FLOPs	**≈ 1.208 GFLOP**
HBM traffic	49.22 MB	24.74 MB	**≈ 73.96 MB**
Arithmetic intensity	≈ 16.36 FLOP/B	≈ 16.28 FLOP/B	**≈ 16.33 FLOP/B**

Duration-based rates

The counter run also records each kernel's duration. Dividing work by duration gives MFMA throughput; dividing bytes by duration gives effective HBM traffic rate.

Kernel	Executed work	Duration	MFMA throughput
9121	805.31 million FLOPs	18.026 µs	**≈ 44.67 TFLOP/s**
9122	402.65 million FLOPs	9.894 µs	**≈ 40.69 TFLOP/s**

Kernel	HBM traffic	Duration	Effective traffic rate
9121	49.22 MB	18.026 µs	**≈ 2.73 TB/s**
9122	24.74 MB	9.894 µs	**≈ 2.50 TB/s**

At this point, the profiler had shown what the hardware executed: the kernels performed matrix arithmetic and moved data between HBM and the GPU.

The remaining question was:

What calculation in the model produced that work?

8. Connect the hardware measurements back to the model

The counters told us how much work the GPU executed. Now we can ask:

How much work did the model actually need?

Qwen3-30B-A3B routes each token to 8 of 128 experts. For each selected expert, the main matrix path is:

token

↓

gate + up

↓

activation

↓

down

Kernel	Operation	Minimum useful FLOPs	Executed FLOPs	Extra-work ratio
9121	Gate/up projection	50,331,648	805,306,368	**16×**
9122	Down projection	25,165,824	402,653,184	**16×**

For one decode token, the useful gate/up work is:

8×1×2048×1536×2=50.33 million FLOPs

But the GPU counters reported:

805.31 million executed FLOPs

For the down projection:

8×1×768×2048×2=25.17 million useful FLOPs

while the GPU executed:

402.65 million FLOPs

In both cases:

executed work≈16×useful work

That is the important result.

ONE TOKEN / EXPERT

Useful model work

↓

~50M FLOPs

Hardware work

↓

~805M FLOPs

↓

16× gap

For one decode token, the useful gate/up work is:

8×1×2048×1536×2=50.33 million FLOPs8 \times 1 \times 2048 \times 1536 \times 2 = 50.33\ \text{million FLOPs}8×1×2048×1536×2=50.33 million FLOPs

But the GPU counters reported:

805.31 million executed FLOPs805.31\ \text{million executed FLOPs}805.31 million executed FLOPs

For the down projection:

8×1×768×2048×2=25.17 million useful FLOPs8 \times 1 \times 768 \times 2048 \times 2 = 25.17\ \text{million useful FLOPs}8×1×768×2048×2=25.17 million useful FLOPs

while the GPU executed:

402.65 million FLOPs402.65\ \text{million FLOPs}402.65 million FLOPs

In both cases:

executed work≈16×useful work\text{executed work} \approx 16 \times \text{useful work}executed work≈16×useful work

That is the important result.

text

```
ONE TOKEN / EXPERT

Useful model work

        ↓

   ~50M FLOPs

Hardware work

        ↓

   ~805M FLOPs

        ↓

      16× gap
```

The profile does not prove exactly why that gap exists. But it gives us a specific mechanism to investigate.

GPU matrix kernels operate on regular blocks of work. With only one useful token reaching each expert during batch-one decode, part of that execution shape may be filled with padding or otherwise underused.

So the next question is no longer:

Why is `fused_moe_kernel` expensive?

It is:

Why does low-batch MoE decode execute 16× the useful matrix arithmetic, and can a different kernel shape or more useful tokens per expert reduce that gap?

That is where this profiling pass ends.

54,194 GPU dispatches

↓

6,144 fused_moe_kernel launches

↓

9121 + 9122 repeated decode kernels

↓

hardware counters

↓

FLOPs + bytes

↓

compare with model math

↓

16× executed / useful arithmetic

↓

specific next experiment